BIRZEIT UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

COMP338 - Artificial Intelligence

Classification by Decision Tree to Identify Most Suitable Crops for Certain Environment

Partners names:

Raheeq Mousa 1220515

Heba Jamal 1212401

Instructor name:

Dr. Abdel-Rahman Hamarsheh

Introduction **3**
Project Overview 3
Dataset Description 3
Methodology **4**
Decision Tree Algorithm 4
Data Preprocessing 4
5-Fold Cross-Validation 4
Model Evaluation 4
Results **5**
Evaluation Metrics (Average Across 5 Folds) 5
Decision Tree Visualization 6
User Input Prediction Example 6
Discussion **7**
Performance Comparison Between Folds 7
Source Code **8**
Conclusion **10**

Introduction

Project Overview

In this project, we aim to develop an AI-based solution to recommend the most suitable crop for a given environment using a Decision Tree algorithm. The model is trained using climatic data for various farmlands, which includes parameters such as soil content, temperature, humidity, and rainfall. The objective is to predict the crop type that thrives best under these conditions.

The dataset used for this project provides real-life data for several farmlands, each labeled with the most suitable crop for the given environmental parameters. Our task is to train a Decision Tree model, validate it using 5-fold cross-validation, and evaluate its performance using accuracy, precision, and recall.

Dataset Description

The dataset contains several columns representing the features (inputs) and one column representing the target label (crop type). The features in the dataset are as follows:

- **N**: Nitrogen content in the soil
- **P**: Phosphorus content in the soil
- **K**: Potassium content in the soil
- **temperature**: Average temperature
- **humidity**: Average relative humidity
- **ph**: Soil pH value
- **rainfall**: Rainfall in mm

The target variable, **label**, represents the crop most suited to the specific environmental conditions (e.g., rice, maize, cotton).

Dataset source: [Kaggle Crop Recommendation Dataset]

Methodology

Decision Tree Algorithm

A Decision Tree is a supervised learning algorithm used for classification tasks. It works by splitting the data into subsets based on feature values. Each split represents a decision point that helps classify the data into the target label. In this project, we used a Decision Tree Classifier from the sklearn library to build the model.

Data Preprocessing

The dataset was cleaned and preprocessed as follows:

- Categorical variables (crop labels) were encoded using LabelEncoder from the sklearn.preprocessing module, which converts string labels into numerical values.

- We ensured that there were no missing values in the dataset, and all features were correctly formatted.

5-Fold Cross-Validation

To evaluate the model's performance reliably, we used 5-fold cross-validation. In this method, the dataset is divided into 5 subsets (folds). The model is trained on 4 of these folds and tested on the remaining one. This process is repeated 5 times, ensuring that each fold is used for testing once.

During each fold, we calculated the following evaluation metrics:

- **Accuracy**: The ratio of correct predictions to the total number of predictions.

- **Precision**: The ratio of true positive predictions to the total predicted positives.

- **Recall**: The ratio of true positive predictions to the total actual positives.

These metrics were calculated for each fold, and the results were averaged to obtain the final performance scores.

Model Evaluation

The Decision Tree model was evaluated using the following metrics:

- **Accuracy**: Provides an overall view of the model's correct predictions.

- **Precision**: Measures how well the model identifies only the correct crops.

- **Recall**: Measures how well the model identifies all suitable crops.

Results

Evaluation Metrics (Average Across 5 Folds)

After performing the 5-fold cross-validation, the following average metrics were obtained:

- **Accuracy**: 0.988%
- **Precision**: 0.989%
- **Recall**: 0.988%

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Samples}}$$

The proportion of total correct predictions (both positive and negative) out of all predictions made. An overall average accuracy of 0.9868 (98.68%) means the model correctly predicts about 98.7% of all

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

The proportion of correctly predicted positive samples out of all samples predicted as positive. An average precision of 0.9874 (98.74%) means when the model predicts a positive class, it's correct 98.74% of the time.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

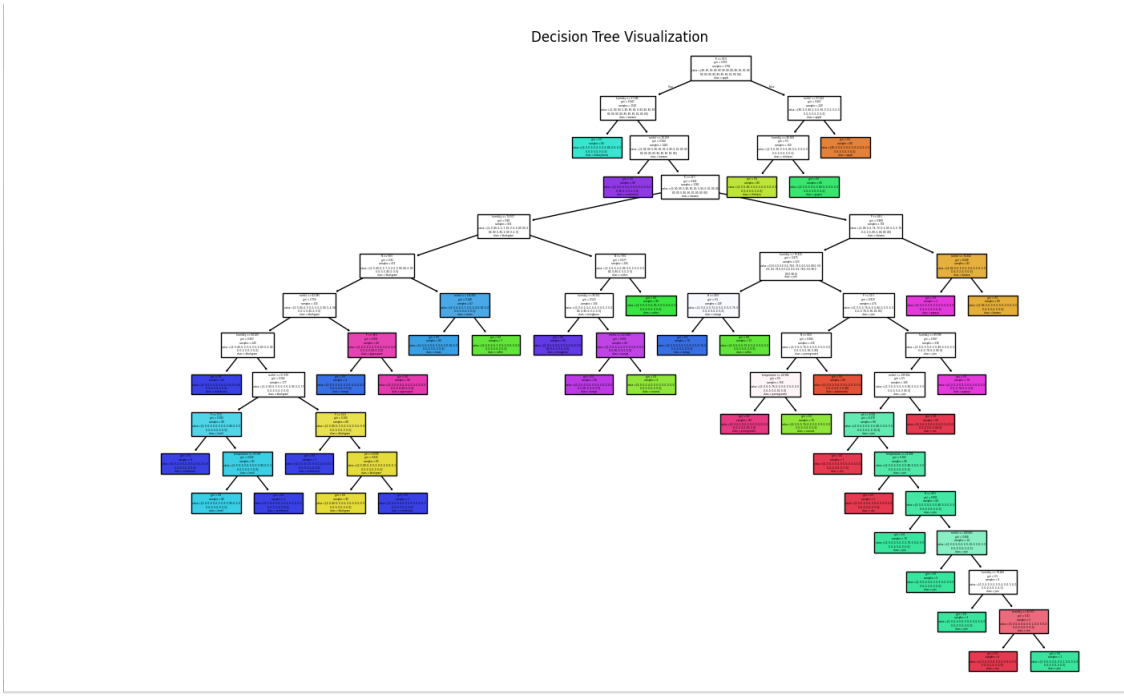
The proportion of correctly predicted positive samples out of all actual positive samples.An overall average recall of 0.9868 (98.68%) means the model finds about 98.68% of all actual positive cases.

The model is both accurate overall and balanced in detecting positive cases without many missing predictions.

The result indicate that the model is both accurate overall and balanced in detecting positive cases without many missing predictions, so it performs well in classifying the most suitable crop for the given environmental conditions.

Decision Tree Visualization

The following decision tree was generated using the trained model. It represents the decision-making process for selecting the most suitable crop based on the input parameters (e.g., Nitrogen, Phosphorus, Temperature, etc.).



User Input Prediction Example

The trained Decision Tree model was used to predict the suitable crop based on user input. For example, when the user inputs the following values:

- Nitrogen: 50
- Phosphorus: 30
- Potassium: 20
- Temperature: 30

- Humidity: 65
- pH: 6.5
- Rainfall: 100

The model recommended **Mango** as the most suitable crop for these conditions.

Discussion

Performance Comparison Between Folds

The model's performance metrics varied slightly between each fold, with accuracy ranging from 97% to 99%. These variations are typical in cross-validation and can be attributed to the differences in training and testing data in each fold.

- **Accuracy**: The average accuracy was high, suggesting that the model is capable of making reliable predictions.
- **Precision**: Precision was also high, indicating that the model was good at correctly identifying crops without making too many false positive predictions.
- **Recall**: The recall was slightly lower than precision, which may suggest that the model sometimes misses certain crop recommendations.

```
--- Fold 1 ---
Accuracy: 0.9795
Precision: 0.9803
Recall: 0.9795
--- Fold 2 ---
Accuracy: 0.9909
Precision: 0.9911
Recall: 0.9909
--- Fold 3 ---
Accuracy: 0.9818
Precision: 0.9834
Recall: 0.9818
--- Fold 4 ---
Accuracy: 0.9795
Precision: 0.9803
Recall: 0.9795
--- Fold 5 ---
Accuracy: 0.9932
Precision: 0.9937
Recall: 0.9932

Overall Average Accuracy: 0.9850
Overall Average Precision: 0.9857
Overall Average Recall: 0.9850
```

All metrics are above 97% which indicates that the model performs well across all subsets of the data.
Small standard deviation in scores, then the model is stable
High precision and recall across the board, then there are low false positives and false negatives
Based on the consistent and high performance metrics across all folds, there is no clear evidence of overfitting in the model. The model generalizes well to unseen data in each fold.

Source Code

We loaded the dataset.

```
dataset = pd.read_csv('Crop_recommendation.csv')
```

Data Preprocessing
convert the categorical labels to numerical values.

```
le = preprocessing.LabelEncoder()
dataset['label'] = le.fit_transform(dataset['label'])
```

Split the data into features and labels.

```
X = dataset.drop(columns='label') #features
y = dataset['label'] #labels
```

Define 5-fold cross-validation.

```
cross_validation = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

Define an arrays to store the metrics across the 5-folds.

```
accuracies = []
precisions = []
recalls = []
```

For each fold iteration we splitted the data into training and testing feature, and training and testing labels.
Then, we used the training data to fit the decision tree into it.
Next, we made prediction on the testing features we splitted before from the dataset.
After that, we calculated the metrics for the decision tree prediction.
Lastly, we appended the metrics to the arrays we defined for tracking the metrics.

```python
fold_num = 1
for train_idx, test_idx in cross_validation.split(X, y): # split the dataset into training and testing sets
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx] # training and testing features
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx] # training and testing labels

    decision_tree.fit(X_train, y_train) # Train the model

    y_pred = decision_tree.predict(X_test) # Make predictions
    # Calculate metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')

    accuracies.append(accuracy)
    precisions.append(precision)
    recalls.append(recall)
# Print metrics for each fold
    print(f"--- Fold {fold_num} ---")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall: {recall:.4f}")
    fold_num += 1
```

Print the Overall Average metrics.

```python
print(f"\nOverall Average Accuracy: {sum(accuracies)/5:.4f}")
print(f"Overall Average Precision: {sum(precisions)/5:.4f}")
print(f"Overall Average Recall: {sum(recalls)/5:.4f}")
```

Plot the decision tree and save it.

```python
# Visualize the Decision Tree
plt.figure(figsize=(100, 50))    "figsize": Unknown word.
plot_tree(
    decision_tree,
    filled=True,
    fontsize=6    "fontsize": Unknown word.
)
plt.savefig("decision_tree_visualization.png", dpi=300, bbox_inches='tight')
plt.title("Decision Tree Visualization", fontsize=20)    "fontsize": Unknown word.
plt.show()
```

This function takes user input and predicts the crop based on the trained model.

```python
# User input prediction
def predict_crop(nitrogen, phosphorus, potassium, temperature, humidity, ph, rainfall):
    # Create a DataFrame for user input
    user_data = pd.DataFrame([[nitrogen, phosphorus, potassium, temperature, humidity, ph, rainfall]], columns=X.columns)
    prediction = decision_tree.predict(user_data)
    crop = le.inverse_transform(prediction)
    return crop[0]
# User input for crop recommendation
print("\n=== Crop Recommendation System ===")
try:
    nitrogen = float(input("Enter Nitrogen content: "))
    phosphorus = float(input("Enter Phosphorus content: "))
    potassium = float(input("Enter Potassium content: "))
    temperature = float(input("Enter Temperature: "))
    humidity = float(input("Enter Humidity: "))
    soil = float(input("Enter Soil pH: "))
    rainfall = float(input("Enter Rainfall: "))

    # Predict the crop based on user input
    recommended_crop = predict_crop(nitrogen, phosphorus, potassium, temperature, humidity, soil, rainfall)
    print(f"\n>>> Recommended Crop: {recommended_crop} <<<")

except ValueError:
    # Handle invalid input
    print("Please enter valid numerical values!")
```

Conclusion

In conclusion, the Decision Tree model performs well in predicting the most suitable crops based on environmental factors. The 5-fold cross-validation method ensured that the model was evaluated rigorously, and the evaluation metrics show that the model is both accurate and reliable. The user input platform allows for real-time predictions, making it a practical tool for farmer.